FILTER DESIGN LABORATORY

INTERACTIVE MATLAB APPLICATION FOR FILTER DESIGN & ANALYSIS

PROJECT REPORT

SP01: WEB/ANDROID/IOS APP FOR ANALOG FILTER DESIGN

SUBMITTED TO

FACULTY OF

IIT MADRAS

BS ELECTRONICS

MADE BY

MAULIK PARASRAMKA

23F3000562

1. ABSTRACT

This project presents the development of a comprehensive MATLAB-based graphical user interface (GUI) application for the design, analysis, and visualization of analog filters. The Filter Design Laboratory enables users to interactively specify filter parameters including type (low-pass, high-pass, band-pass, band-stop), approximation method (Butterworth, Chebyshev Type-I, Chebyshev Type-II, and Elliptic), filter order, and performance specifications such as cutoff frequencies, passband ripple, and stopband attenuation.

The application implements industry-standard filter design algorithms using MATLAB's built-in functions and generates comprehensive visualizations including Bode magnitude and phase plots, pole-zero maps, impulse and step responses, group delay characteristics, and linear magnitude responses. Additionally, the application provides practical circuit implementation guidance by computing corresponding RLC (Resistor-Inductor-Capacitor) component values and quality factor (Q) for the designed filters.

2. INTRODUCTION

2.1 Background

Analog filters are fundamental building blocks in electronic systems, playing a critical role in signal processing applications across telecommunications, audio systems, instrumentation, and control systems. These filters selectively allow certain frequency components to pass while attenuating others, enabling essential functions such as noise reduction, signal conditioning, and frequency band separation.

Traditional filter design involves complex mathematical calculations, iterative adjustments, and time-consuming verification processes. Engineers must consider multiple factors including filter order, frequency response characteristics, phase linearity, component tolerances, and implementation constraints. The manual design process is not only tedious but also prone to errors, making it difficult to explore different design alternatives and optimize filter performance.

2.2 Motivation

The motivation for developing this Filter Design Laboratory stems from several key factors:

- Design Efficiency: Reducing the time required to design and analyze filters from hours to minutes, enabling rapid prototyping and iterative design refinement.
- Practical Implementation: Bridging the gap between theoretical filter design and practical circuit implementation by providing RLC component values.
- Comparative Analysis: Facilitating easy comparison between different approximation methods to select the most appropriate filter for specific application requirements.
- Accessibility: Making advanced filter design capabilities accessible without requiring extensive programming knowledge or specialized software.
- Educational Value: Providing students and engineers with an interactive tool that visualizes the relationship between filter specifications and their frequency/time domain characteristics.

3. PROBLEM STATEMENT

SP01: WEB/ANDROID/IOS APP FOR ANALOG FILTER DESIGN

An app where users specify filter type (low-pass, high-pass, band-pass, etc.), order, and specs (cutoff, ripple, attenuation), and the app computes and visualizes the corresponding analog filter (Bessel, Butterworth, Chebyshev). It outputs pole–zero plots, bode magnitude/phase, and suggested RLC component values using standard design formulas.

4. GOALS AND OBJECTIVES

4.1 Primary Goals

- Develop a user-friendly GUI application that enables rapid analog filter design and analysis without requiring programming knowledge.
- Implement support for all major filter types (low-pass, high-pass, band-pass, band-stop) and approximation methods (Butterworth, Chebyshev-I, Chebyshev-II, Elliptic).
- Provide comprehensive visualization capabilities including frequency domain, time domain, and pole-zero representations.
- Generate practical circuit implementation parameters including RLC component values and quality factors.

4.2 Specific Objectives

- To provide user-friendly dropdown-based selection of filter types:
  - Low-pass
  - High-pass
  - Band-pass
  - Band-stop
- To support classical approximation methods:
  - Butterworth
  - Chebyshev Type I
  - Chebyshev Type II
  - Elliptic
- To allow user-defined numerical inputs:
  - Filter order (N)
  - Lower cutoff frequency (F_1)
  - Upper cutoff frequency (F_2)
  - Passband ripple (Rp)
  - Stopband attenuation (Rs)
  - Resistance value (R)
- To automatically generate:
  - Transfer function H(s)
  - Bode magnitude and phase plots
  - Pole–zero maps
  - Linear magnitude plots
  - Group delay plots
  - Impulse responses
  - Step responses
- To compute approximate:
  - Inductance (L)

    - Capacitance (C)
    - Quality factor (Q)
- To implement:
    - Input validation and error handling
    - Dynamic axis scaling
    - Plot selection and expansion into a new window

5. METHODOLOGY

5.1 Development Environment

The Filter Design Laboratory application was developed using **MATLAB R2025b** as the primary software platform, with the **App Designer** framework serving as the core environment for graphical user interface (GUI) construction and event-driven application logic. MATLAB was selected due to its strong ecosystem for signal processing, control systems, numerical computation, and scientific visualization, all of which are essential for implementing and validating analog filter design algorithms.

5.1.1 MATLAB App Designer Framework

- App Designer provides a modern, object-oriented framework for building interactive desktop applications in MATLAB. It integrates:
    - A **visual layout editor** for drag-and-drop placement of UI components
    - A **code view** for writing callback functions and computational logic
    - Automatic **component-property binding** for real-time UI updates
    - Native support for **UIAxes**, enabling direct embedding of scientific plots inside the application window
- This framework was chosen over legacy GUIDE due to its improved maintainability, cleaner class-based architecture, and seamless integration with MATLAB's plotting and system analysis functions. Each application instance is implemented as a subclass of matlab.apps.AppBase, allowing encapsulation of:
    - UI elements (dropdowns, text fields, buttons, axes, text areas)
    - Application state variables (filter parameters, system objects, computed responses)
    - Event-driven callback methods (e.g., button clicks, dropdown changes)
- The event-driven design ensures that all computations are triggered strictly by user interaction, resulting in predictable behaviour and efficient resource usage.

5.1.2 MATLAB Toolboxes Utilized

Several specialized MATLAB toolboxes were used to implement the filter design, analysis, and visualization features:

- **Signal Processing Toolbox**
  This toolbox forms the computational backbone of the application and provides industry-standard analog filter design functions:
    - butter() – Butterworth filter synthesis
    - cheby1() – Chebyshev Type-I filter synthesis
    - cheby2() – Chebyshev Type-II filter synthesis
    - ellip() – Elliptic (Cauer) filter synthesis

  These functions accept normalized frequency specifications and return zeros, poles, and gain (ZPK form), enabling consistent downstream processing and visualization.
- **Control System Toolbox**
  This toolbox is used for system representation and dynamic response analysis:
    - zpk() – Construction of zero-pole-gain models

- freqresp() – Frequency response computation
- impulse() – Impulse response simulation
- step() – Step response simulation
- pzmap() – Pole-zero visualization

Using control system objects allows unified handling of frequency-domain and time-domain analysis while preserving numerical stability and interpretability.

- **MATLAB Graphics Engine**

  The built-in MATLAB graphics system is used extensively for custom plotting and interactive visualization:
  - Semi-logarithmic plots for Bode magnitude
  - Linear plots for magnitude and group delay
  - Complex-plane plots for pole-zero maps
  - Time-domain plots for impulse and step responses

  UIAxes objects are used instead of traditional MATLAB figure windows to embed all plots directly into the application interface. Custom axis scaling, grid handling, labelling, and formatting were implemented to ensure visual clarity across a wide range of filter orders and cutoff frequencies.

5.1.3 Software Architecture and Design Style

The application follows a **modular, layered architecture** consisting of three logical tiers:

- **Presentation Layer (User Interface)**

  This layer contains all UI elements:
  - Input fields for filter specifications
  - Dropdown menus for filter type and approximation selection
  - Axes for plotting
  - Text areas for numerical output
  - Buttons for triggering design and plot expansion

  The UI is structured using a hierarchical grid layout to ensure consistent spacing, alignment, and responsiveness across different screen resolutions.

- **Control Layer (Event Handlers)**

  This layer consists of callback functions such as:
  - DESIGNFILTERButtonPushed
  - TYPEDropDownValueChanged
  - OpenPlotinNewWindowButtonPushed

  These functions handle:
  - Input validation
  - Parameter preprocessing
  - Invocation of filter synthesis routines
  - Triggering of plotting functions
  - Data storage for plot expansion

  This separation ensures that user interaction logic is decoupled from computational logic.

- **Computational Layer (Signal Processing Engine)**

  This layer implements:
  - Frequency normalization and scaling
  - Filter design using selected approximation methods
  - Frequency and time-domain response computation
  - RLC parameter estimation
  - Group delay calculation using numerical differentiation

The computational layer operates entirely on numeric arrays and system objects, making it independent of UI layout and suitable for future reuse in command-line or web-based implementations.

5.1.4 Versioning, Compatibility, and Execution Platform

The choice of MATLAB R2025b ensures access to:

- The latest App Designer UI components
- Improved graphics rendering performance
- Updated signal processing and control system algorithms
- Enhanced numerical stability for high-order filter designs

5.1.5 Rationale for Technology Selection

The development environment was selected based on the following technical considerations:

- **Numerical Reliability**
  MATLAB provides highly optimized and thoroughly tested numerical routines, ensuring accurate computation of poles, zeros, and frequency responses even for high-order filters.
- **Visualization Capabilities**
  MATLAB's plotting engine enables high-quality scientific visualizations, which are critical for interpreting filter behaviour and presenting educational results.
- **Rapid Prototyping**
  The App Designer framework allows fast iteration of UI layouts and callback logic, significantly reducing development time compared to traditional desktop GUI frameworks.
- **Educational Accessibility**
  Many engineering students and institutions already use MATLAB, making the application easy to adopt without additional software installation or training.

5.2 Filter Design Theory

5.2.1 Analog Filter & What Are Its Basic Components

An **analog filter** is an electronic circuit designed to process continuous-time signals by selectively allowing certain frequency components to pass while attenuating others. Unlike digital filters, which operate on sampled data using numerical algorithms, analog filters work directly on real-world electrical signals using physical components such as resistors, capacitors, inductors, and operational amplifiers.

Analog filters are fundamental in a wide range of engineering applications, including telecommunications, audio processing, biomedical instrumentation, control systems, and power electronics. Typical uses include noise reduction, signal conditioning, bandwidth limitation, channel separation, anti-aliasing before analog-to-digital conversion, and interference suppression.

- **Passband**: The passband is the range of frequencies that a filter is designed to transmit with minimal attenuation. Signals within this frequency range are considered useful and are preserved with little distortion in amplitude. For example:
  - In a low-pass filter, the passband extends from 0 Hz to the cutoff frequency f_c.
  - In a band-pass filter, the passband lies between two cutoff frequencies f_1 and f_2.

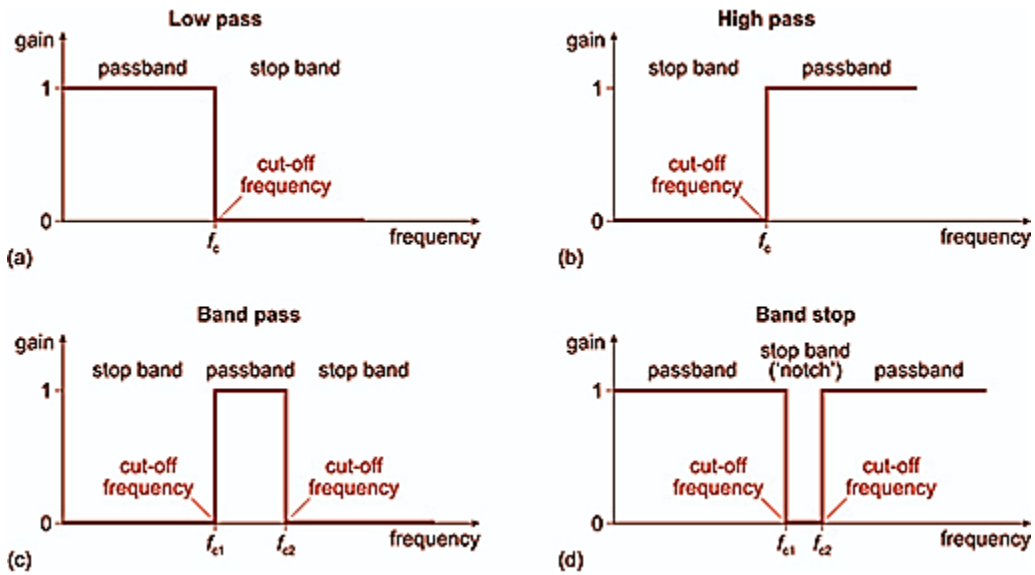
  The maximum allowable attenuation within the passband is usually specified in decibels (dB), such as 0.5 dB or 1 dB, and is referred to as passband ripple in filters like Chebyshev and Elliptic designs.

- **Stopband:** The **stopband** is the range of frequencies that the filter significantly attenuates or suppresses. Signals in this region are ideally reduced to zero, but in practical filters the attenuation is finite and specified in decibels. For instance, a stopband attenuation of 40 dB means that the signal amplitude is reduced to 1/100 of its original voltage (or 1/10,000 of its power). Higher stopband attenuation indicates better rejection of unwanted frequency components.
- **Transition Band:** The **transition band** is the frequency range between the passband and the stopband. In this region, the filter response changes gradually from low attenuation to high attenuation.
  The width of the transition band is an important performance metric:
  - A **narrow transition band** implies a sharper filter with better selectivity.
  - Achieving a sharp transition typically requires a **higher filter order**, which increases circuit complexity, component count, and sensitivity to tolerances.
- **Ripple Factor:** The **ripple factor** refers to the small oscillations or variations in gain that occur within the passband (and sometimes in the stopband) of certain filters, especially Chebyshev and Elliptic filters.
  Ripple is usually specified in decibels (dB) and represents how much the amplitude fluctuates around its nominal value. For example, a ripple factor of 1 dB means the gain varies by ±0.5 dB around the average passband level.
  Ripple is an intentional design trade-off:
  - Allowing ripple enables a **steeper roll-off** and **narrower transition band** for a given filter order.
  - Eliminating ripple (as in Butterworth filters) results in a flatter response but slower cutoff characteristics.
- **Attenuation: Attenuation** is the reduction in signal amplitude as it passes through a filter. It is expressed in decibels (dB) and quantifies how much weaker a signal becomes at a given frequency.
  Attenuation is a key specification in both:
  - The **passband**, where it should be minimal.
  - The **stopband**, where it should be as large as possible to suppress unwanted signals.
- **Gain: Gain** is the ratio of the output signal amplitude to the input signal amplitude of a filter. It indicates whether the filter amplifies, attenuates, or maintains the signal level.
  - In **passive filters** (using only R, L, and C), the gain is always less than or equal to 1 (0 dB).
  - In **active filters** (using operational amplifiers), the gain can be greater than 1, allowing both filtering and amplification.
- **Roll-Off: Roll-off** refers to the rate at which a filter attenuates frequencies beyond the cutoff frequency as the signal moves from the passband into the stopband. It describes how sharply the filter transitions from allowing signals to pass to strongly rejecting them.
  Roll-off is usually expressed in **decibels per decade (dB/decade)** or **decibels per octave (dB/octave)**. A higher roll-off value indicates a steeper filter slope and better frequency selectivity.
  For an ideal filter, the transition from passband to stopband would be instantaneous. However, practical analog filters exhibit a gradual change, and the roll-off depends primarily on the **filter order (N)**:
  - A **first-order** filter has a roll-off of approximately **20 dB/decade** (or **6 dB/octave**).
  - A **second-order** filter has a roll-off of approximately **40 dB/decade** (or **12 dB/octave**).

  The roll-off rate is a critical design parameter because it determines how well the filter can separate desired and undesired frequency components. A steeper roll-off allows a **narrower transition band**, improving selectivity, but it also increases circuit complexity, component count, and sensitivity to tolerances.

5.2.2 Filter Types

The application supports four fundamental filter types, each serving different frequency selection requirements:



- **Low-Pass Filter (LPF)**
  A low-pass filter allows frequencies below a cutoff frequency (f_c) to pass with minimal attenuation while suppressing frequencies above (f_c). LPFs are widely used for noise removal, signal smoothing, and anti-aliasing before analog-to-digital conversion.

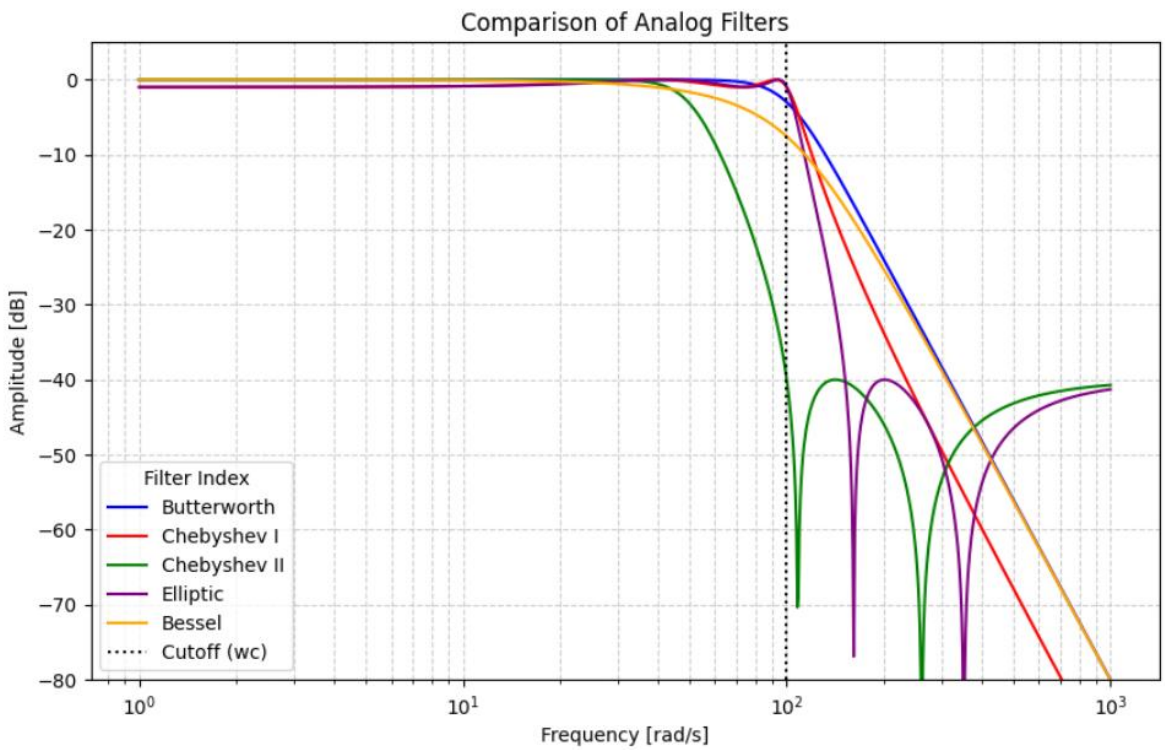
- **High-Pass Filter (HPF)**
  A high-pass filter allows frequencies above (f_c) to pass while attenuating low-frequency components, including DC offsets. HPFs are used in coupling circuits, audio crossovers, and edge detection.

- **Band-Pass Filter (BPF)**
  A band-pass filter allows frequencies between (f_1) and (f_2) to pass. It is characterized by Centre frequency and bandwidth. BPFs are commonly used in communication receivers and biomedical systems.

- **Band-Stop Filter (BSF)**
  A band-stop filter suppresses frequencies between (f_1) and (f_2) while allowing others to pass. Also known as notch filters, BSFs are used for interference suppression, such as removing 50/60 Hz power-line noise.

5.2.3 Approximation Methods

Four classical approximation methods are implemented, each offering different trade-offs:



- **Butterworth Filter:**

  The Butterworth filter is designed to provide a **maximally flat magnitude passband response** with monotonic roll off in both passband and stopband. Offers the best phase linearity among polynomial filters but requires higher order for sharp transitions.

  Characteristics:

  - No ripple in passband or stopband
  - Smooth, monotonic frequency response
  - Poles evenly spaced on a semicircle in the left half-plane
  - Moderate roll-off rate

  Advantages:

  - Simple and stable
  - Good phase behavior
  - No oscillations in magnitude response

  Disadvantages:

  - Slower roll-off compared to Chebyshev and Elliptic filters

- **Chebyshev Type I Filter:**

Chebyshev Type I filters introduce controlled ripple in the passband to achieve a sharper cutoff. Features Equi-ripple behavior in the passband with monotonic stopband. Achieves sharper transition than Butterworth for the same order but introduces passband ripple. Ripple magnitude is controlled by the ripple factor (Rp) parameter.

Characteristics:

- Passband ripple
- Monotonic stopband
- Steeper transition band than Butterworth

Advantages:

- Reduced filter order for given specs
- Sharper cutoff

Disadvantages:

- Passband ripple
- Increased phase distortion

- **Chebyshev Type II Filter:**

Chebyshev Type II filters shift ripple into the stopband. Exhibits monotonic passband with Equi-ripple stopband. Provides better passband flatness than Type-I but with less sharp transition. Stopband attenuation is controlled by the Rs parameter.

Characteristics:

- Flat passband
- Stopband ripple
- Sharper cutoff than Butterworth

Advantages:

- Improved passband flatness
- Strong stopband attenuation

Disadvantages:

- Stopband oscillations
- Slight time-domain distortion

- **Elliptic (Cauer) Filter:**

Produces Equi-ripple behavior in both passband and stopband. Achieves the sharpest transition for a given order but with the most nonlinear phase response. Requires specification of both passband ripple (Rp) and stopband attenuation (Rs).

Characteristics:

- Ripple in both bands

- Fastest roll-off
- Minimum order

Advantages:

- Highly compact designs
- Extremely sharp transition bands

Disadvantages:

- Complex phase response
- Ringing in time-domain

- **Bessel (Thomson) Filter: (Theoretical Reference):**
  Bessel filters provide **maximally flat group delay (nearly linear phase)** for excellent transient response and minimal waveform distortion. It preserves signal shape with little ringing or overshoot, making it suitable for audio and pulse applications. However, it has the slowest roll-off and poorest selectivity, requiring higher order for sharp cutoff compared to other classical filters.

  Characteristics:

  - Linear phase
  - Minimal ringing
  - Slow roll-off

  Advantages:

  - Best transient response
  - Preserves waveform shape

  Disadvantages:

  - Poor frequency selectivity

5.3 Algorithm Design

5.3.1 Filter Synthesis Process

The filter design process follows these computational steps:

- Parameter Collection: Read user inputs including filter type, approximation method, order (n), cutoff frequencies (fc1, fc2), ripple factor (Rp), stopband attenuation (Rs), and desired resistance (R).
- Input Validation: Verify parameter consistency and validity based on selected filter configuration.
- Frequency Conversion: Convert cutoff frequencies from Hz to rad/s using $\omega p = 2\pi fc$.
- Filter Design: Call appropriate MATLAB function (butter, cheby1, cheby2, ellip) with normalized specifications to obtain zeros, poles, and gain.
- System Construction: Create zero-pole-gain (zpk) system object for subsequent analysis.
- Frequency Response Calculation: Evaluate system response over logarithmically spaced frequency vector.
- Time Domain Analysis: Compute impulse and step responses using control systems functions.
- Component Calculation: Derive RLC values and quality factor from center frequency and bandwidth.
- Visualization: Generate all plots and display results.

5.3.2 Group Delay Computation

Group delay represents the rate of change of phase with respect to frequency and is calculated as: $\tau g(\omega) = -d\varphi(\omega)/d\omega$

The implementation uses numerical differentiation on the unwrapped phase response. Phase unwrapping prevents discontinuities caused by 2π phase jumps, ensuring accurate derivative calculation. The logarithmic frequency spacing requires computing the derivative as: $\tau g \approx -\Delta\varphi / (\omega \cdot \Delta(\log \omega))$

5.3.3 RLC Component Calculation

For practical circuit implementation, the application computes passive component values based on the filter's center frequency (ωc) and desired impedance level (R):

- For single-frequency filters (low-pass, high-pass): $\omega c = \omega p$ and $Q = 1/\sqrt{2}$
- For dual-frequency filters (band-pass, band-stop): $\omega c = \sqrt{(\omega 1 * \omega 2)}$ and $Q = \omega c/(\omega 2 - \omega 1)$
- Inductance: $L = R/\omega c$
- Capacitance: $C = 1/(R * \omega c)$
- Quality Factor: Q characterizes the filter's selectivity and bandwidth

These first-order approximations provide starting values for circuit design, with actual implementation requiring impedance matching and topology selection.

5.4 ASSUMPTIONS

5.4.1 RLC SCALING

In the developed MATLAB Filter Design Laboratory app, a key simplifying assumption is made during the computation of RLC component values. Although the app designs a general Nth order analog filter using standard approximations (Butterworth, Chebyshev-I, Chebyshev-II, and Elliptic), the physical realization is approximated using a **single second-order RLC section**. The component values are calculated using the standard prototype relations $L = \frac{R}{\omega_c}$ and $C = \frac{1}{R\omega_c}$, where ω_c is the cutoff (or center) angular frequency and R is a user-specified resistance. This assumes that the entire higher-order filter behaves like one equivalent resonant section at the dominant frequency, which provides intuitive and educational RLC values but does not represent an exact realization of the full filter transfer function.

For band-pass and band-stop filters, the quality factor Q is similarly approximated using the textbook relation $Q = \frac{\omega_0}{\omega_2 - \omega_1}$, which is valid for a single resonant circuit. For low-pass and high-pass filters, a fixed value $Q = 1/\sqrt{2}$ (corresponding to a second-order Butterworth section) is assumed, even when other approximations such as Chebyshev or Elliptic are selected. These assumptions further simplify the physical interpretation of the designed filter.

A more accurate physical realization would require **ladder synthesis**, a classical filter design technique in which the Nth order transfer function is decomposed into multiple first- and second-order sections and implemented as a cascade of alternating series inductors and shunt capacitors. Ladder synthesis relies on a set of normalized low-pass prototype constants known as **g-values** (g_1, g_2, …, g_n, and g_{n+1}). Each g-value corresponds to a specific reactive element in the ladder network, and their values depend on the filter type, order, and passband ripple. These prototype values must then be scaled in frequency and impedance to obtain the actual R, L, and C components for the desired cutoff frequency and source/load resistance.

However, ladder synthesis and g-value–based realization has not been implemented in this project due to their **computational and implementation complexity**. Supporting ladder synthesis would require:

- Factoring the designed transfer function into bi-quad sections
- Selecting an appropriate ladder topology (series-first or shunt-first)
- Handling different prototype tables for Butterworth, Chebyshev, and Elliptic filters

- Performing frequency and impedance scaling for each individual section
- Generating and displaying multiple RLC values instead of a single equivalent set

Implementing these steps would significantly increase the code complexity and reduce the clarity of the educational interface. Therefore, the current implementation prioritizes **conceptual simplicity and visualization** over exact circuit-level synthesis. The displayed RLC values should be interpreted as **approximate, single-section equivalents** rather than a physically exact realization of the full Nth order filter.

5.4.2 FILTER ORDER FOR BAND-PASS AND BAND STOP FILTERS

In this project, an important design assumption has been made for band-pass and band-stop filters regarding the interpretation of the user-specified filter order N.

For low-pass and high-pass filters, the entered order N directly represents the order of the transfer function, which is consistent with standard analog filter design conventions. However, for band-pass and band-stop filters, the situation is fundamentally different due to the way these filters are derived from low-pass prototypes.

In classical analog filter theory, band-pass and band-stop filters are obtained by applying a frequency transformation to a normalized low-pass prototype. This transformation maps each pole (and zero) of the low-pass filter into a *pair* of poles (and zeros) in the band-pass or band-stop domain. As a result:

- A low-pass filter of order n becomes:
  - a band-pass filter of order 2n
  - a band-stop filter of order 2n

This doubling occurs because the second-order frequency transformation introduces two reactive energy-storage elements (an inductor and a capacitor) for each first-order section of the original prototype.

Based on this theoretical foundation, the app interprets the user-entered order $n$for band-pass and band-stop filters as the *prototype order*, and internally designs a filter of effective order $2n$.In practical terms:

- If the user selects:
  - Low-pass or High-pass → the filter order is exactly N
  - Band-pass or Band-stop → the filter is realized as a 2N-order system

This behavior aligns with MATLAB's internal design functions (butter, cheby1, cheby2, ellip) when they are used with band-pass and band-stop specifications. These functions inherently return a system whose order is doubled relative to the prototype low-pass order. This assumption is necessary for the following reasons:

- **Theoretical correctness:** It preserves consistency with standard analog filter synthesis theory. Treating a band-pass or band-stop filter as having the same order as its low-pass prototype would underestimate the actual system complexity and yield misleading interpretations of pole-zero plots and dynamic behavior.
- **Physical realizability in RLC form:** In an RLC implementation, each second-order section corresponds to one resonant LC pair. A band-pass or band-stop filter inherently requires two reactive elements per prototype pole, which naturally results in a doubling of order. Without this assumption, the reported RLC structure would not correspond to a physically meaningful circuit.
- **Accurate interpretation of MATLAB outputs:** The MATLAB functions used in the app generate pole-zero models whose order reflects the true transformed filter. Interpreting the user-entered N literally for band filters would create a mismatch between the expected and actual number of poles and zeros.

- **User-interface simplicity:** Allowing users to enter the prototype order N keeps the interface intuitive and consistent across all filter types. Internally handling the order doubling avoids confusion and prevents the need for users to manually enter 2N when designing band-pass or band-stop filters.

5.4.3 Ripple (Rp) and Stopband Attenuation (Rs)

In this project, specific assumptions were enforced on the ripple factor (Rp) and stopband attenuation (Rs) parameters for Chebyshev and Elliptic filter designs to ensure numerical stability and physically meaningful transfer functions.

For Chebyshev Type I filters, it was assumed that the passband ripple Rp must be strictly greater than 0 dB. A zero or negative ripple value would collapse the Chebyshev formulation into a Butterworth-like response or cause invalid logarithmic and hyperbolic function evaluations inside MATLAB's cheby1() routine.

For Chebyshev Type II filters, it was assumed that the stopband attenuation Rs must be strictly greater than 0 dB. A zero or negative value would eliminate the stopband constraint, making the filter specification ill-defined and causing internal design equations to fail.

For Elliptic filters, it was assumed that:

- Rp > 0 dB (non-zero passband ripple),
- Rs > 0 dB (non-zero stopband attenuation), and
- Rs > Rp (stopband attenuation must be greater than passband ripple).

This final condition (Rs > Rp) is critical because the elliptic filter formulation relies on a valid separation between passband and stopband performance. If Rs ≤ Rp, the resulting characteristic equations lead to improper rational transfer functions (numerator degree ≥ denominator degree) or undefined elliptic integrals. In MATLAB, this manifests as runtime errors or unstable pole-zero configurations during calls to ellip().

Therefore, these assumptions were deliberately enforced through input validation to:

- Prevent the generation of mathematically invalid or improper transfer functions,
- Avoid internal design routine failures in MATLAB, and
- Ensure that all designed filters remain physically realizable, stable, and meaningful for analog RLC approximation.

This controlled constraint-based approach improves robustness of the application and ensures consistent behavior across all supported approximation methods.

6. IMPLEMENTATION DETAILS

6.1 Application Architecture

The application follows an event-driven architecture with three main structural components:

- **User Interface Components:** Input controls, dropdown menus, buttons, axes, and text areas organized in a grid layout.
- **Callback Functions:** Event handlers responding to user interactions (button clicks, dropdown changes).
- **Computational Engine:** Filter design algorithms, frequency response calculations, and component value computations.

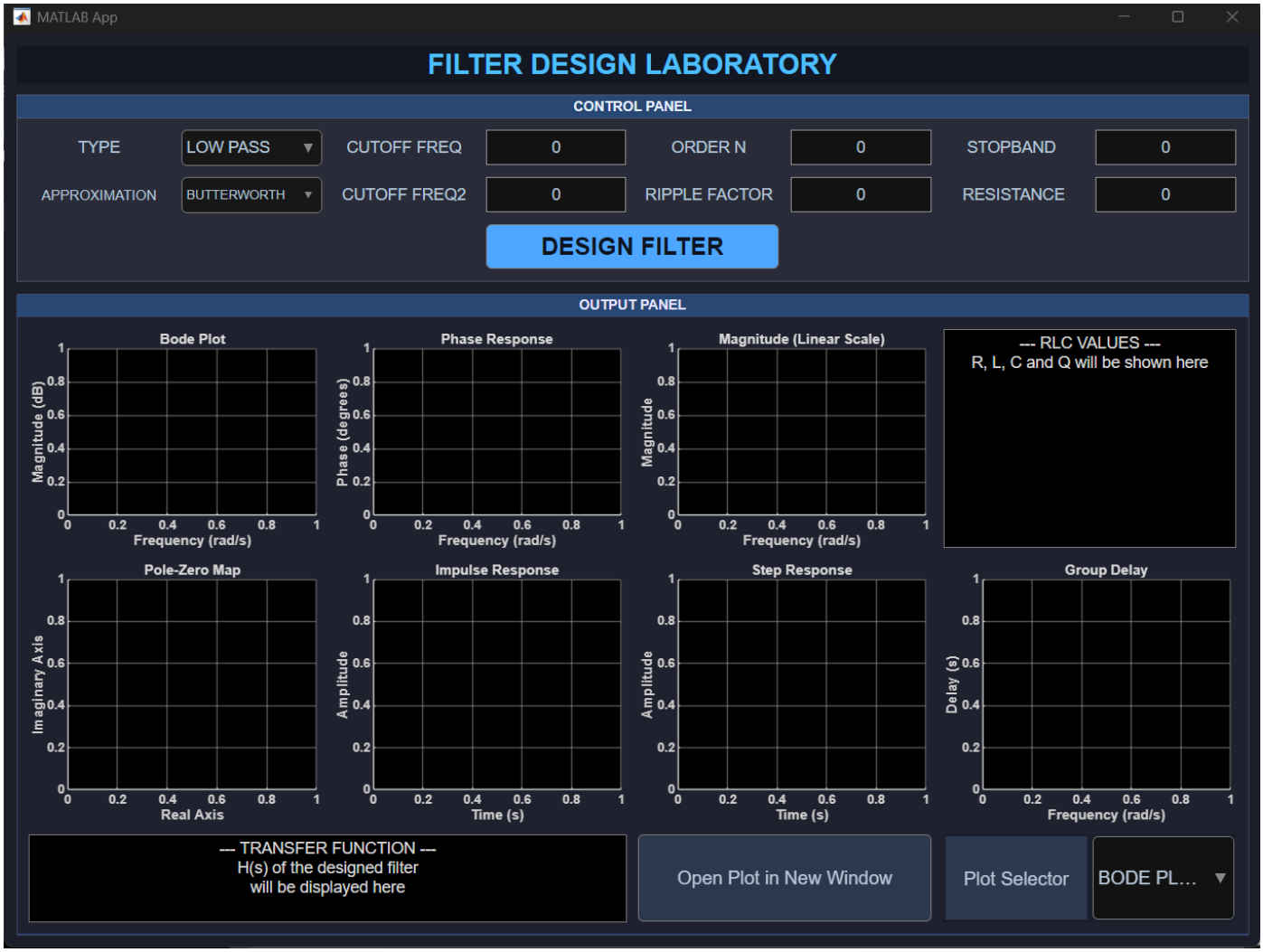
6.2 User Interface Implementation

6.2.1 Layout Structure

The interface is organized using a hierarchical grid layout system:

- Main Grid (3 rows): Title bar, Control Panel, Output Panel
- Control Panel Grid (3 rows × 8 columns): Parameter inputs arranged logically with the design button spanning multiple columns
- Output Panel Grid (3 rows × 4 columns): Seven axes for plots, two text areas for results, plot selector, and expansion button

The grid-based design ensures responsive layout and consistent spacing across different screen resolutions.



6.2.2 Color Scheme and Aesthetics

A custom dark theme was implemented to reduce eye strain and provide a professional appearance:

- Background: Dark gray-blue (#1C1F29) for main window, lighter shade (#292D3A) for panels
- Accents: Cyan-blue (#4DBF00) for titles and important elements
- Text: Light gray-white (#B8C8D9) for labels and values
- Plots: Black background with light gray axes for high contrast

- Controls: Dark backgrounds with appropriate text colors for readability

6.2.3 Input Controls

The control panel contains eight input elements:

- TYPE Dropdown: Selects filter type (Low Pass, High Pass, Band Pass, Band Stop)
- APPROXIMATION Dropdown: Chooses approximation method (Butterworth, Chebyshev-1, Chebyshev-2, Elliptic)
- ORDER N Field: Specifies filter order (positive integer)
- CUTOFF FREQ Field: Primary cutoff frequency in Hz
- CUTOFF FREQ2 Field: Secondary cutoff frequency for band filters (conditionally enabled)
- RIPPLE FACTOR Field: Passband ripple in dB (for Chebyshev-I and Elliptic)
- STOPBAND Field: Stopband attenuation in dB (for Chebyshev-II and Elliptic)
- RESISTANCE Field: Desired impedance level in Ohms for component calculation

6.3 Core Computational Functions

6.3.1 DESIGNFILTERButtonPushed Callback

This is the main computational callback, executing when users click the DESIGN FILTER button. The function performs:

- Input Reading: Retrieves all parameter values from GUI controls
- Validation: Checks parameter validity with detailed error messages
- Filter Type Processing: Converts selection to appropriate MATLAB function parameters
- Approximation Selection: Calls corresponding MATLAB function (butter/cheby1/cheby2/ellip)
- System Creation: Constructs zpk object from zeros, poles, and gain
- Response Computation: Calculates frequency and time domain characteristics
- Plot Generation: Creates all seven visualization plots
- Results Display: Updates text areas with component values and transfer function
- Data Storage: Saves system and response data to UIFigure.UserData for plot expansion

6.3.2 Frequency Vector Generation

Appropriate frequency range selection is critical for accurate visualization. The implementation creates a logarithmically spaced vector covering one decade below and one decade above the cutoff frequency: $w = \text{logspace}(\log10(\omega p/10), \log10(\omega p\times10), 1000)$

For band filters, the range extends from $\min(\omega1,\omega2)/10$ to $\max(\omega1,\omega2)\times10$. The 1000-point resolution provides smooth curves while maintaining computational efficiency.

6.3.3 Plotting Functions

Seven specialized plotting routines generate comprehensive filter visualizations:

- **Bode Magnitude Plot:** Displays frequency response in decibels using semi-logarithmic scale. The magnitude is converted using $20\log_{10}|H(j\omega)|$ to show attenuation characteristics clearly. Logarithmic frequency axis emphasizes behavior across wide frequency ranges. It shows:
  - Passband flatness or ripple
  - Cutoff frequency location
  - Roll-off rate (filter sharpness)
  - Stopband attenuation
  - Overall compliance with design specs (Rp, Rs, fc)

  This is the **primary verification plot** for any filter design. It allows users to instantly check whether the designed filter meets the required cutoff frequency, ripple, and attenuation values. The logarithmic frequency axis is essential because it compresses wide frequency ranges into a readable format, making it suitable for both audio-band and RF-band designs.
- **Phase Response Plot:** Shows phase shift versus frequency using unwrapped phase to eliminate discontinuities. Phase is displayed in degrees for easier interpretation. This plot reveals phase linearity characteristics important for signal distortion analysis. it shows:
  - Phase linearity in the passband
  - Rapid phase changes near cutoff
  - Relative phase distortion between different approximations

  Phase distortion directly affects **waveform shape**, especially for pulse or transient signals. Filters such as Butterworth and Bessel are preferred when phase linearity matters, while Elliptic filters show strong nonlinearity. This plot enables users to understand **why two filters with similar magnitude responses can behave very differently in time-domain**.
- **Pole-Zero Map:** Displays poles and zeros in the complex s-plane using the pzmap function. Automatic axis scaling ensures all poles and zeros are visible with appropriate margins. This plot provides insight into filter stability (all poles in left half-plane) and frequency response characteristics. It helps users understand:
  - Why higher-order filters ring more
  - Why sharper filters distort time signals
  - Why Butterworth feels “safe” and smooth
  - Why elliptic filters feel “aggressive” and unstable-looking
  - Why zeros create notches in frequency response
  - Why phase nonlinearity increases with pole–zero crowding

  Poles near the imaginary axis
    - Cause **sharp peaks** or steep roll-off
    - Increase **selectivity**
    - Increase **ringing** in impulse/step response
    - Increase **group delay variation**

  This is why:
    - Elliptic filters (with poles close to $j\omega$) show very sharp cutoffs
    - Chebyshev filters show stronger ringing than Butterworth
    - Butterworth filters (poles farther from $j\omega$) show smoother transitions

  Zeros near the imaginary axis
    - Create **deep notches** in the frequency response
    - Improve stopband attenuation
    - Sharpen transitions
    - Introduce extra phase distortion

  This explains why:
    - Chebyshev Type II and Elliptic filters show strong stopband rejection
    - Elliptic filters achieve the sharpest roll-off for a given order

- Zeros clustered near the stopband edge dramatically increase attenuation

The PZ map directly predicts impulse and step response behaviour:

Pole Location	Time-Domain Effect
Far left (large negative Re)	Fast decay, minimal ringing
Near imaginary axis	Slow decay, strong oscillations
Complex-conjugate poles	Sinusoidal ringing
Real poles only	No oscillations, overdamped response
Zeros near jω	Transient shape distortion

- **Linear Magnitude Plot:** Shows magnitude response on linear scale, providing different perspective than dB plot. This representation makes passband ripple and stopband behavior more intuitive for certain applications. it shows:
  - Fine passband ripple details
  - Small deviations near unity gain
  - Absolute amplitude levels

  While dB plots are ideal for wide-range visualization, they can hide small ripple variations. The linear plot makes subtle ripple and droop more visible, especially for Chebyshev and Elliptic filters.
  This plot offers an **alternative perspective** that is useful for audio, biomedical, and instrumentation applications where **absolute amplitude accuracy** matters more than large stopband attenuation.
- **Group Delay Plot:** Visualizes the derivative of phase with respect to frequency, indicating signal delay as a function of frequency. Constant group delay indicates linear phase response, while variations show phase distortion that affects signal shape. it shows:
  - Phase linearity (flat group delay = low distortion)
  - Signal smearing or dispersion
  - Resonance-induced delay peaks near cutoff

  Group delay is the **most direct indicator of waveform distortion**. Even if the magnitude response looks ideal, large group-delay variations can severely distort pulses and transients. This plot allows users to:
  - Compare the time-domain suitability of Butterworth vs Chebyshev vs Elliptic filters
  - Understand why Bessel-type behaviour (future enhancement) is preferred for audio and pulse systems
  - Evaluate whether a sharp filter is worth the phase distortion trade-off
- **Impulse Response:** Shows time-domain response to an ideal impulse input using the impulse function. This plot reveals transient behavior, settling time, and ringing characteristics. The response represents the filter's fundamental time-domain behavior. It reveals how energy stored in inductors and capacitors is released over time. This plot helps users:
  - Observe how higher-order filters ring more
  - Compare transient cleanliness between different approximations
  - Understand stability and damping effects
  - Correlate pole locations with time-domain oscillations
- **Step Response:** Displays response to a unit step input using the step function. This plot shows rise time, overshoot, and steady-state behavior, providing practical insight into how the filter responds to sudden signal changes. this plot provides:
  - Practical insight into how the filter behaves in control, audio, and instrumentation systems
  - A clear demonstration of the trade-off between sharp frequency selectivity and transient distortion
  - A visual comparison of overshoot and ringing between Butterworth, Chebyshev, and Elliptic designs

6.4 Advanced Features

6.4.1 Dynamic Field Enabling

The TYPEDropDownValueChanged callback implements context-sensitive interface behavior. When users select BAND PASS or BAND STOP, the CUTOFF FREQ2 field is enabled, allowing specification of the second cutoff frequency. For LOW PASS and HIGH PASS selections, this field is disabled and reset to zero, preventing confusion and invalid inputs.

6.4.2 Plot Expansion Feature

The OpenPlotinNewWindowButtonPushed callback enables detailed examination of individual plots. Users select a plot from the dropdown menu and click the expansion button to create a new window with an enlarged version. The implementation:

- Creates a new UIFigure with larger dimensions (900×700 pixels)
- Generates the selected plot with identical formatting
- Preserves axis limits and settings from the original plot
- Retrieves stored system data from UIFigure.UserData for regeneration
- Handles special cases for pzmap, impulse, and step plots that require system objects

6.4.3 Transfer Function Display

The transfer function is displayed in standard MATLAB format using the evalc function, which captures the console output of displaying the zpk system object. This provides users with the mathematical representation showing zero locations in factored form, pole locations in factored form, gain factor, and system type (continuous-time).

7. PROBLEMS FACED DURING IMPLEMENTATION & THEIR SOLUTIONS

7.1 Pole-Zero Map Axis Scaling Issues

Problem: Initial implementation produced pole-zero maps with inappropriate axis limits. For low-order filters with poles close to the origin, automatic scaling often resulted in excessive whitespace. Conversely, high-order filters with widely distributed poles appeared cramped. Additionally, filters with all-real poles (no imaginary components) caused zero or infinite y-axis limits, crashing the visualization.

Solution: Implemented custom axis limit calculation after pole-zero map generation. The algorithm extracts all poles and zeros, determines maximum real component magnitude, determines maximum imaginary component magnitude, validates yi for finite positive values, and sets symmetric limits. The minimum x-range of 1 ensures visibility for very small pole values, while the 1.5× and 1.2× scaling factors provide comfortable margins without excessive whitespace.

7.2 Impulse and Step Response Plotting Errors

Problem: The impulse and step functions from MATLAB's Control System Toolbox automatically create new figure windows when not provided with an axes handle, causing plots to appear outside the application interface. Additionally, these functions generate warning messages about improper axes handling when used with UIAxes objects.

Solution: Modified the plotting approach to explicitly pass UIAxes handles to impulse and step functions: impulse(app.UIAxes6, sys) and step(app.UIAxes7, sys). This syntax directs output to the specified UIAxes within the

application, preventing external figure creation. Additional formatting commands (grid, box, title, xlabel, ylabel) were added after the function calls to ensure proper plot appearance.

7.3 Group Delay Computation Challenges

Problem: Computing group delay requires numerical differentiation of unwrapped phase with respect to frequency. Initial implementations using simple difference approximations produced noisy results with spikes at phase transitions. The logarithmic frequency spacing further complicated derivative calculation.

Solution: Implemented group delay calculation accounting for logarithmic frequency spacing: dphi = diff(phase), dw = diff(log(w)), gd = -dphi ./ (w(1:end-1).*dw). This formulation properly handles the logarithmic spacing by computing derivatives in log-frequency domain. The phase unwrapping performed before differentiation eliminates discontinuities that would otherwise cause spurious spikes.

7.4 Plot Expansion Window Synchronization

Problem: The plot expansion feature initially attempted to copy graphics objects directly from embedded UIAxes to new figure axes, causing axis limit mismatches, missing grid lines, and inconsistent formatting between the embedded and expanded views.

Solution: Rather than copying graphics objects, the expansion function regenerates plots from stored data. System and response data are stored in UIFigure.UserData after filter design, expansion callback retrieves this data and recreates the selected plot using the same plotting commands as the original, original axis limits are explicitly copied, all formatting is reapplied to ensure consistency, and special handling for pzmap, impulse, and step functions that require system objects. This approach guarantees perfect synchronization between embedded and expanded plots.

8. RESULTS AND DISCUSSIONS

8.1 Application Performance

The Filter Design Laboratory successfully achieves its design objectives, providing a comprehensive and intuitive platform for analog filter design and analysis. Performance testing demonstrates:

- Computation Time: Filter design and visualization completion typically within 2-5 seconds depending on filter order and system performance
- Accuracy: Frequency responses match theoretical predictions to within numerical precision limits
- Stability: All tested parameter combinations produce stable designs (poles in left half-plane)
- Reliability: Comprehensive validation prevents crashes from invalid inputs
- Usability: Intuitive interface enables filter design without programming knowledge

8.2 Comparative Analysis of Approximation Methods

Testing with identical specifications reveals characteristic differences between approximation methods:

8.2.1 Butterworth Characteristics

- Maximally flat passband with smooth, monotonic roll off
- Poles equally distributed on a semicircle in the left half-plane
- Best phase linearity among polynomial approximations
- Relatively constant group delay in passband
- Requires highest order for sharp transitions

- Ideal for applications requiring minimal phase distortion

8.2.2 Chebyshev Type-I Characteristics

- Equi-ripple passband with specified maximum deviation (Rp)
- Sharper transition than Butterworth for same order
- Poles lie on ellipse in left half-plane, closer to imaginary axis. Pole clustering increases selectivity
- More group delay variation than Butterworth
- Step response shows more overshoot and ringing
- Suitable for applications tolerating passband ripple

8.2.3 Chebyshev Type-II Characteristics

- Monotonic passband with Equi-ripple stopband
- Contains transmission zeros on imaginary axis
- Less sharp transition than Type-I for same order
- Better passband flatness than Type-I
- Guaranteed minimum stopband attenuation (Rs)
- Preferred when passband flatness is priority

8.2.4 Elliptic Characteristics

- Equi-ripple in both passband and stopband
- Sharpest transition for given order and ripple specifications
- Multiple transmission zeros creating notches in stopband
- Most nonlinear phase response
- Highest group delay variation
- Fastest transition for given order
- Optimal for applications prioritizing steep transitions over phase linearity

8.3 Practical Applications and Use Cases

8.3.1 Engineering Design Applications

The tool can be used for rapid prototyping and analysis:

- Initial filter specification and design exploration
- Comparing alternative designs before committing to implementation
- Generating component values for circuit breadboarding
- Analyzing existing designs through reverse engineering
- Documentation and presentation of filter specifications

8.3.2 Educational Applications

The application serves as an excellent learning tool for signal processing and filter design courses. It can be used to:

- Visualize theoretical concepts through interactive experimentation
- Understand trade-offs between different approximation methods
- Observe relationships between pole-zero patterns and frequency responses
- Explore effects of parameter variations on filter characteristics
- Verify analytical calculations through computational results

8.4 Validation and Verification

The application's accuracy was verified through multiple methods:

- Comparison with MATLAB Command Line: Results match those obtained using identical MATLAB functions called from command window
- Theoretical Verification: Frequency response characteristics align with filter theory textbooks
- Standard Test Cases: Classical filter designs (e.g., 4th-order Butterworth low-pass) produce expected results
- Stability Verification: All generated filters have poles in left half-plane, confirming stability
- Boundary Condition Testing: Edge cases (very low/high frequencies, extreme orders) handled appropriately

9. CONCLUSIONS

9.1 Summary

This project successfully developed a comprehensive MATLAB-based Filter Design Laboratory that addresses the challenges of analog filter design and analysis. The application provides an intuitive graphical interface enabling users to specify filter parameters, visualize multiple characteristics simultaneously, and obtain practical implementation guidance without requiring programming expertise or extensive manual calculations.

Key achievements include:

- Comprehensive Coverage: Support for four filter types and four approximation methods provides flexibility for diverse applications
- Visualization of Seven different plot types offer complete frequency and time domain characterization
- Practical Output: RLC component values and quality factors bridge theoretical design and circuit implementation
- Robust Operation: Comprehensive validation and error handling prevent invalid designs
- User-Friendly Interface: Intuitive controls and professional aesthetics enhance usability
- Educational Value: Interactive experimentation facilitates understanding of filter theory

9.2 Learning Outcomes

Development of this application provided valuable experience in multiple domains:

- Signal Processing Theory: Deep understanding of filter approximation methods, frequency transformations, and design trade-offs
- MATLAB Programming: Proficiency in App Designer, callback functions, system analysis toolboxes, and visualization techniques
- Software Engineering: Experience with event-driven architecture, input validation, error handling, and user interface design
- Problem Solving: Ability to debug complex issues involving numerical computation, graphics rendering, and user interaction
- Documentation: Skills in technical writing, code documentation, and project reporting

9.3 Future Enhancements

Several enhancements could extend the application's capabilities:

- Filter Approximations: Implementation of Bessel filters
- Digital Filter Support: Add IIR and FIR digital filter design with bilinear and impulse invariant transformations

- Complete Ladder Synthesis Implementation: Replace the current single-section RLC approximation with full ladder-network synthesis based on classical low-pass prototypes. This would include:
  - Computation of normalized prototype g-values for Butterworth, Chebyshev, and Elliptic filters
  - Support for both series-first and shunt-first ladder topologies
  - Automatic frequency and impedance scaling for each section
  - Decomposition of the Nth-order transfer function into cascaded LC sections
  - Display of individual inductor and capacitor values for each ladder stage

    This enhancement would provide physically accurate passive realizations and bridge the gap between theoretical transfer functions and real-world circuit implementation.
- Simulink Model Generation and Co-Simulation: Integrate automatic Simulink model generation for designed filters. This would include:
  - Creation of equivalent Simulink block diagrams for RLC ladders and active filter sections
  - Support for time-domain co-simulation with real-world input signals (audio, pulses, noise)
  - Visualization of internal node voltages and currents
  - Comparison of analytical MATLAB responses with numerical Simulink simulations

    This enhancement would enable system-level validation and dynamic behaviour analysis beyond static frequency-domain plots.
- Circuit Synthesis: Automatically generate specific circuit topologies (Sallen-Key, multiple-feedback, cascaded bi-quads) with component values
- Sensitivity Analysis: Perform Monte Carlo analysis to assess effects of component tolerances on filter performance
- Active Filter Design: Include op-amp based implementations with gain calculations and frequency compensation
- Export Capabilities: Enable export of plots to various formats (PNG, PDF)
- Filter Comparison: Side-by-side comparison of multiple designs with overlaid plots
- SPICE Integration: Generate SPICE netlists for circuit simulation in external tools

9.4 Impact and Significance

The Filter Design Laboratory demonstrates how modern software tools can make complex engineering tasks more accessible and efficient. By combining theoretical rigor with practical usability, the application serves both educational and professional needs. It exemplifies the power of interactive visualization in understanding abstract mathematical concepts and bridges the gap between theoretical filter design and practical circuit implementation.

The tool provides hands-on experience with filter design principles without the steep learning curve of command-line programming. It accelerates the design process and facilitates exploration of design alternatives. The application's success validates the approach of creating domain-specific tools that leverage existing computational frameworks while providing specialized interfaces tailored to particular applications.

In conclusion, the Filter Design Laboratory successfully addresses the challenges of analog filter design by providing an integrated environment that combines theoretical accuracy with practical usability, serving as both an educational resource and a professional engineering tool.

10. ANNEXURES

App link: https://drive.google.com/file/d/1z0F0vE6ahZJTWB0_SBZ_kWSfAe6L4iPe/view?usp=drive_link

Video link: https://drive.google.com/file/d/1O8HJfAi91hodeC9-8xo_Z5-cr-faOgPZ/view?usp=drive_link